

Securing Data Pipelines for AI Systems

AI data pipelines move data into prompts, RAG indexes, and fine-tuning sets, and move model output back out. Each hop crosses a confidentiality boundary. Three controls keep sensitive data contained: data loss prevention, confidentiality boundaries, and least-privilege access.

Data loss prevention (DLP)

Detect and classify sensitive data **before** it enters the model boundary (prompt, index, training set) and **before** output leaves it. Redact or block by policy.

```
from airte.data_pipeline import DLPEngine, Classification
eng = DLPEngine()
eng.max_classification("SSN 123-45-6789")      # -> Classification.RESTRICTED
eng.redact("contact a@b.com, AKIA...")        # -> "[REDACTED:email], [REDACTED:aws_key]"
```

The engine ships high-precision regex detectors (SSN, credit card with Luhn check, email, phone, IPv4, AWS keys, API keys) and a 4-level classification (PUBLIC → INTERNAL → CONFIDENTIAL → RESTRICTED). For context-dependent PII that regex misses (names, locations), plug in the optional **Microsoft Presidio** analyzer behind the same interface: `DLPEngine(analyzer=get_analyzer("auto"))` uses Presidio if installed and transparently falls back to regex otherwise. Use it at two choke points: **ingest** (gate what enters a RAG index / fine-tune set) and **egress** (redact model answers and tool inputs).

Confidentiality boundaries

- **Classify data and pin it to a boundary.** RESTRICTED data should never enter a shared prompt, a multi-tenant index, or a model whose outputs aren't redacted.
- **Per-tenant isolation.** Separate vector namespaces/indexes per tenant; never let one tenant's query retrieve another's vectors (defeats cross-tenant leakage, LLM08).
- **Minimize what reaches the model.** Send the least data needed; prefer references or tool-fetch-at-execution over stuffing sensitive records into context.
- **Redact at the boundary, not just at rest.** A record may be fine in the database but must be redacted before it becomes prompt context or model output.

Least-privilege access for LLMs

The classic failure is the **confused deputy**: the LLM/agent has broader read access than the end user, so a crafted request retrieves data the user shouldn't see. Bind retrieval and tool access to the *requesting user's* entitlements.

```
from airte.data_pipeline import DataScope, evaluate_access, Classification
from airte.data_pipeline.least_privilege import AccessRequest
```

```
scope = DataScope(tenants=frozenset({"acme"}),
                  max_classification=Classification.CONFIDENTIAL,
                  purposes=frozenset({"support"}))
evaluate_access(scope, AccessRequest("acme", Classification.INTERNAL, "support")) # ok
evaluate_access(scope, AccessRequest("other", Classification.INTERNAL, "support")) # raises
```

`evaluate_access` enforces tenant isolation, a classification ceiling, and **purpose limitation** (data used only for the stated purpose) — the same principles as data- protection regimes, expressed as code.

Pipeline architecture

```
source ■■ DLP classify/redact (ingest) ■■ per-tenant index ■■ retrieval scoped to
  user's entitlements ■■ fence as untrusted data ■■ model ■■ DLP redact (egress) ■■
  schema-validate ■■ downstream
```

- **Provenance & integrity.** Record source per chunk; HMAC-sign records on ingest and verify before use to detect poisoning/tampering (LLM04/LLM08). See `airte.data_pipeline.provenance` (`SignedRecord.create / .verify`).
- **Encryption + key management.** Encrypt at rest and in transit; manage keys in a vault (see `cloud-security-ai.md`).
- **Audit.** Log what data was retrieved, for whom, and what was redacted, with a correlation ID tying it to the model interaction.

Combine with `airte.guardrails.rag_guardrails` (fencing/quarantine) so that even data that legitimately enters the context cannot smuggle instructions.